

Redis & BullMQ no Pipeline de RAG

Processamento Assíncrono e Ingestão de Documentos em Escala

1. Introdução Geral

Este documento descreve detalhadamente o funcionamento e a utilidade prática do Redis e do BullMQ no ecossistema do Lumina LMS. O processamento de arquivos PDF ou de texto para o RAG (Retrieval-Augmented Generation) exige etapas computacionais pesadas: extração de texto bruto, fragmentação inteligente (chunking), comunicação com APIs de inteligência artificial da OpenAI (geração de embeddings) e gravação de vetores no PostgreSQL (pgvector).

Para evitar gargalos na interface do usuário (timeouts na rota de API) e garantir que o sistema continue extremamente ágil, implementou-se uma arquitetura baseada no padrão de projeto Producer-Consumer (Produtor-Consumidor), viabilizada pela combinação robusta de Redis e BullMQ.

🔑 O CONCEITO CHAVE (PRODUCER-CONSUMER):

Quando o professor faz o upload de um material, a rota de API apenas valida o arquivo, salva-o localmente e registra a tarefa no Redis (Producer). A requisição termina instantaneamente para o usuário final, e um processo independente (Consumer/Worker) é notificado para realizar a extração e vetorização do documento em segundo plano, sem travar o servidor principal.

2. O Fluxo de Ingestão de Dados Passo a Passo

Passo 1: Upload e Salvamento Físico

O usuário envia o arquivo (PDF/TXT) pela interface gráfica. A rota POST `/api/modules/[moduleId]/sources` recebe o `multipart/form-data`, valida o tamanho e formato, aciona o `StorageService` e salva o arquivo na pasta `./storage/uploads/modules/[moduleId]/[timestamp].[pdf|txt]`.

Passo 2: Registro e Disparo do Processamento

Logo em seguida, o sistema registra o arquivo na tabela `ModuleSourceFile` no Postgres com status "UPLOADED" e dispara a chamada para a service `processSourceFile()`. Esta service atualiza o status para "PROCESSING" e adiciona um Job no BullMQ através de `enqueueDocumentProcessing(fileId, moduleId)`.

Passo 3: Fila no Redis (O Papel do Redis + BullMQ)

O Redis serve como a engrenagem central da fila. O BullMQ cria uma estrutura de dados de fila chamada "embedding-processing" dentro do Redis. Os Jobs enviados guardam o ID do arquivo e do módulo. O Redis armazena de forma persistente a fila e gerencia concorrências, garantindo que o job nunca seja perdido mesmo se o servidor cair.

Passo 4: Processamento no Worker em Segundo Plano

Um Worker dedicado (`embeddingWorker`) configurado no arquivo `embedding-worker.ts` fica rodando em segundo plano ouvindo a fila no Redis. Ao detectar o novo Job:

1. O Worker extrai o texto do arquivo (PDF ou texto simples) usando o `document-parser`.
2. Divide o texto em fragmentos inteligentes de até 500 tokens (Token-Aware Chunking).
3. Envia os fragmentos para a API da OpenAI (`text-embedding-3-small`) para computar os embeddings vetoriais de 1536 dimensões.
4. Limpa chunks antigos e salva os novos chunks e seus respectivos vetores na tabela `ModuleSourceChunk` no PostgreSQL (utilizando o `pgvector`).
5. Atualiza o status do arquivo original para "PROCESSED" no banco de dados.

❗ POR QUE ESTA ARQUITETURA É ALTAMENTE RECOMENDADA?

- **Evita Request Timeout:** O parseamento de PDFs longos e a geração de embeddings na OpenAI podem demorar dezenas de segundos. Em servidores Next.js convencionais (como na Vercel), requisições HTTP normais possuem timeout de 10 a 15 segundos. A fila assíncrona resolve isso completamente.
- **Resiliência Automática:** Se a API da OpenAI falhar devido a instabilidade, o BullMQ automaticamente retenta processar o Job até 3 vezes com um tempo de espera exponencial (backoff exponencial de 2 segundos).
- **Controle de Concorrência e Escala:** O Redis impede sobrecargas. Você pode subir 5 workers separados em servidores diferentes para rodar o processamento paralelamente sem interferir na rota de navegação do app.

3. Onde os Códigos de Redis e BullMQ estão de fato?

- **docker-compose.yml**

Contém a definição do container Docker "lumina_redis" rodando a imagem leve "redis:7-alpine" na porta padrão "6379", garantindo que a infraestrutura suba localmente de forma simples com "docker compose up -d".

- **src/lib/env.ts**

Valida e injeta a variável de ambiente REDIS_URL utilizando a biblioteca Zod, configurando "redis://localhost:6379" como fallback caso a variável não seja informada.

- **src/lib/rag/jobs/queues/embedding-queue.ts**

Cria a instância do RedisConnection e define a fila "embedding-processing" do BullMQ. Exporta a função assíncrona enqueueDocumentProcessing() para injetar tarefas de forma simples no ecossistema.

- **src/lib/rag/jobs/workers/embedding-worker.ts**

O motor em segundo plano. Declara o "embeddingWorker" ouvindo a fila no Redis. Possui toda a lógica de parseamento, chunking, geração de embeddings da OpenAI e persistência no banco vetorial via pgvector.

- **src/lib/rag/jobs/workers/start-workers.ts**

Código bootstrap responsável por inicializar os Workers do BullMQ, ouvir eventos de progresso, sucesso ou erro, além de gerenciar o encerramento gracioso (Graceful Shutdown) fechando a conexão com o Redis em caso de SIGINT.

CONFIGURAÇÕES DE REDIS NO ARQUIVO .ENV

A conexão física do projeto com a instância do Redis é estabelecida puramente a partir de variáveis de ambiente. Abaixo está a listagem exata de como a infraestrutura de filas e o Redis estão configurados no arquivo de ambiente:

- REDIS_URL="redis://localhost:6379"
- UPLOAD_DIR="./storage/uploads"
- RAG_CHUNK_SIZE="500"
- RAG_CHUNK_OVERLAP="80"

Isto significa que para rodar o pipeline assíncrono em ambiente local, basta subir o container correspondente e executar o Next.js normalmente com "npm run dev".